

InVision

Team Guide - Task Division and Viva Explanations

The system is built on **two models**: CLIP, which encodes what a storefront looks like, and RapidOCR, which reads what its signage says. Everything else in the project is one of three jobs performed around those two models - preparing what they see, fusing what they output, and acting on the result.

That gives the team a natural three-way division. Each member owns one job end to end, can explain the whole system through it, and has their own design decisions to defend and their own demonstration to run.

Job	Owns	One-line summary
1. FEED	Data and perception	Prepare the input both models depend on
2. FUSE	Identity and recognition	Combine both model outputs into an answer
3. ACT	Navigation and application	Turn that answer into directions

Read your own section closely and the other two once. Evaluators routinely ask each member about the seams between their part and the next.

1. The system in one page

Every member should be able to give this summary, whichever job they own. It is the frame the rest of the guide hangs on.

The two models

	CLIP ViT-B/32	RapidOCR
Reads	What the shopfront looks like	What the shopfront says
Output	512-d normalised vector	Text, confidence, bounding box
Training	None - used zero-shot	None - off the shelf
Strength	Works when text is unreadable	Names the shop outright
Weakness	Confuses adjacent lookalike units	Loses characters inside logo marks

State plainly that **we trained nothing**. That is a design decision, not a shortcut, and Section 5 gives the full defence.

Why two and not one

The models fail on different photographs, and that is the entire reason both are present. Two outlets of one brand look alike *and* read alike, so neither model separates them alone - but they sit at different positions on different floors. A shopfront caught at a steep angle may read poorly yet still match visually. A shop that was never photographed cannot be matched visually at all, yet its sign is perfectly legible. Section 5 measures this: fusing the two beats the better single model by more than fifteen points.

The three jobs built around them

- **FEED** - the photographs arrive unlabelled, so this job recovers the structure hidden in them and shapes what each model receives: the walk order, the floor, and the text-height signal that separates a brand name from a discount poster.
- **FUSE** - both models score every candidate shop, and this job decides the answer from those two scores, including which model wins when they disagree.
- **ACT** - a recognised shop is only useful if it leads somewhere, so this job builds the mall's navigation graph and the application that delivers directions.

The sentence that ties it together

Two models supply the evidence; the three jobs decide what evidence they get, what the evidence means, and what to do about it.

2. Job one - FEED: data and perception

Ours: `src/ingest.py`, `src/encoders.py`, `src/grouping.py`, `src/pipeline.py`, `build_dataset.py`, and the `tools/scripts`.

What to say

My part is what both models depend on. The constraint that shaped every decision is this: we captured 289 photographs of the mall and **not one of them carries a store label**. We did not start with a dataset, we started with a folder of pictures. So the first job was to find whatever structure already existed in the data.

There were exactly two usable signals. The folder name - ours read *1st Left*, *2nd Right* - gives the floor and which side of the corridor we walked. And the timestamp inside each filename gives something more valuable than it first appears: sort a folder by timestamp and you recover **the exact order in which we walked the corridor**. A walk passes shops in sequence, so that ordering later becomes the evidence that lets us place photographs against the floor map. It is free information that was sitting in the filenames.

One detail that would silently break the whole app: the mall labels its ground floor as *1st*, so every folder ordinal is one level too high. We map each ordinal down by one, making folder *1st* floor zero. Without that single line of config, every route sends the user to the wrong floor.

Preparing input for the CLIP model

Each photograph is encoded into a 512-dimensional vector, L2-normalised so that cosine similarity reduces to a plain dot product - which keeps recognition fast, because scoring the whole gallery is one matrix multiplication. Images are batched, and any file that fails to decode is recorded rather than silently dropped, so row alignment between the vectors and the image list is never lost.

Preparing input for the OCR model

This is the most important decision in my section. RapidOCR returns three things per detection: the text, a confidence, and a bounding box. **We keep the height of that box**, then express it as a fraction of the tallest text in the same photograph.

The reason is the key insight of the project. On a shopfront, **the brand name is the largest lettering on the facade**. Promotional copy - SALE, 50% OFF, NEW ARRIVALS - is set far smaller. And critically, that promotional copy repeats across a store's frames just as often as the brand does, so counting how often a word appears cannot separate them. Relative text height can. Our brand ranking is therefore **50% relative height, 30% frequency across the store's frames, 20% OCR confidence** - height dominates deliberately.

The same height signal is reused twice more downstream, which is worth pointing out: it is how the system decides which of two shopfronts in a wide photograph is the one you are actually standing in front of.

I also filter vocabulary that must never name a store - generic mall words such as EXIT and ESCALATOR, sale wording in several languages because our facade posters use SOLDES and REBAJAS, and importantly **the mall's own branding**: NEXUS, MALL, SELECT. Those appear on every corridor. Left unfiltered, the mall's own name wins as the brand for half the stores.

A number I want to be honest about

Where OCR reads nothing, CLIP similarity between consecutive frames acts as a fallback, with a cutoff of 0.80. **I calibrated that rather than guessing it** - there is a dedicated script - and the honest finding is that the margin is thin: the tightest genuinely matching pair scores 0.812, barely above the line. That is exactly why signage evidence is primary and appearance is the fallback. I would rather report a narrow margin than present a fragile number as solid.

What my stage produces

Quantity	Value
Images ingested	289
Images with readable signage	288
Gallery vectors (dimension 512)	234
Held-out query vectors	55
Stores recovered	83

Demonstration: run `build_dataset.py` and walk through the generated `report.md`, which states at the top that every count in it is derived rather than ground truth.

Questions to expect

Why CLIP rather than ResNet or a fine-tuned backbone?

CLIP is trained on image-text pairs from the web, so its embedding space already separates brands and shopfront styles with no fine-tuning. A shopfront is largely defined by its logo and signage, which is exactly the content CLIP has seen most of. It also runs adequately on CPU, which matters for a laptop demonstration.

Why not train a classifier on the images instead?

We have roughly three images per shop. A classifier needs many samples per class; with three it memorises rather than generalises, and every new tenant would require retraining the model.

How do you know the OCR output is reliable?

We do not claim it is - we designed around its failure modes. It reliably loses leading characters absorbed into a logo mark: ESTSIDE for WESTSIDE, DASICS for ASICS. The matching stage is built to tolerate exactly that.

Why keep stores that have only one photograph?

A store needs at least three images before it can give one up for evaluation without crippling its own gallery. Below that we keep everything and flag it. Eight stores are in that position.

3. Job two - FUSE: identity and recognition

Owns: src/directory.py, src/frame_align.py, src/build_stores.py, src/localizer.py, evaluate.py, predict.py, and the review tool.

What to say

My part decides what the two models' outputs actually mean. Both models score every candidate shop; my job is to turn those two scores into one answer, including deciding which model wins when they disagree.

The most useful thing I can tell you is that **our first approach failed, we diagnosed why, and we rebuilt it.**

The failure, and how we proved it

Originally we leaned on the CLIP model to group the photographs: consecutive frames that looked alike were clustered into one shop, and each cluster was then named from its signage. The flaw is structural. Whenever two neighbouring shopfronts looked similar, the appearance model merged them - and a merged cluster can only carry one name. A food court collapsed four stalls into one, and the other three ceased to exist in our system.

We proved that rather than assuming it. I searched the raw OCR output for the brands the pipeline claimed were never photographed. **Fourteen of fifteen were sitting there in the signage text.** So the photographs existed and the information was lost in grouping, not in capture. That diagnostic is what justified the rewrite.

The rewrite: let the OCR model lead

The fix follows directly from the two models' different strengths. Appearance similarity says only that two frames *resemble* each other, which is a weak claim about identity when neighbouring shops share a design language. Signage says what a shop *is*. So the OCR model leads and the CLIP model corroborates - the reverse of what we started with.

Concretely, photographs are matched straight onto the units listed in the official floor directory, using the walk order recovered in job one. Frames where the OCR model reads a fascia cleanly and unambiguously settle their own identity outright; the frames around them - interiors, crowds, promotional boards, wayfinding signs, everything whose text is actively misleading - inherit identity from those confident neighbours instead of inventing one of their own. Walk order constrains the whole assignment, because a corridor is walked in sequence, and anything claiming to break that sequence is discarded. In practice what gets discarded is exactly what you would want: reflections in glass, and directory boards naming a shop on the far side of the mall. Where the OCR model reads nothing at all, CLIP similarity keeps consecutive frames together so the sequence still holds.

The principle to state in one line: **a frame that can identify itself does; a frame that cannot borrows from the nearest one that can.**

Matching noisy text to real brand names

Comparing an OCR string against an official brand name is a sequence alignment problem, so we use Needleman-Wunsch with a deliberately mild gap penalty. Mild, because skipped and merged units are *expected* - the alignment should tolerate them rather than force bad pairings. Substring containment scores highly, specifically because the characteristic OCR failure is losing leading characters. And we keep an alias table for what fuzzy matching can never solve: MARKSSPENCER and M&S share almost

no characters, so only a listed alternate spelling connects them.

Fusing the two models at query time

When a user submits a photograph, the CLIP model scores it against every unit's reference images by best match, and the OCR model scores its signage against every unit's official name. The two are combined at a default weight of **0.7 visual to 0.3 text**.

One rule overrides that fusion, and it was added after we saw real misrecognitions. **If the signage in the query clearly names one unit and beats the runner-up by a margin, that decides the answer outright.** The logic is blunt - if the sign says ALDO, it is Aldo, regardless of which reference photograph happens to look closest. This matters for two reasons: a shop with no reference images scores zero on the visual channel however plain its sign, and a mislabelled gallery actively pulls the answer toward the shop next door.

The subtle bug worth describing in the viva

Originally the text channel matched against words harvested from a unit's own gallery images. That is self-defeating. If a group is labelled ALDO but actually holds photographs of the shop next door, its vocabulary fills with the neighbour's brand - so a genuine photo of that neighbour scores highest against the wrong shop. **The mislabelling was teaching itself.** Matching against the directory name instead breaks the feedback loop, because the directory is independent of how our photographs were grouped.

Where two fascias are both legible - common in wide survey shots - we do not silently guess. The result is reported as a close call and the rival reading is kept directly behind the winner, so the interface can offer it as a one-tap correction.

Finally, we built a human review tool: a page showing every frame in walk order with its assigned unit, which a reviewer can correct. Those corrections are treated as ground truth and beat anything the system infers - because everything the system produces is a guess about which shopfront a photograph shows, whereas a reviewer has actually looked.

Questions to expect

Why is the directory allowed to override your model? Is that not cheating?

The directory is external knowledge about the world, not evaluation data. It tells us which shops exist and in what order. It does not tell us which photograph is which - that remains the models' job.

Why add OCR at all if CLIP already works?

Because the two fail on different images, which the ablation demonstrates: fusion beats the better single model by more than fifteen points. Text also reaches shops with no reference photographs, which visual retrieval fundamentally cannot.

Your accuracy dropped after an improvement. Explain that.

The benchmark grades against automatically generated labels. When we fixed the localiser so a Tommy Hilfiger photograph is named Tommy Hilfiger, the stored label still said Calvin Klein, so a correct answer was counted wrong. We can demonstrate this on specific images. It is exactly why we built the review tool: the metric only becomes meaningful once the labels are verified.

Every direction downstream would be wrong, so the system reports confidence rather than hiding it. When the top two candidates are close it says so and offers the alternative as a one-tap correction.

4. Job three - ACT: navigation and application

`Owns: src/mall_graph.py, routes.py, api.py, web/index.html.`

What to say

The models tell us where the user is standing. My part decides where everything in the mall is, and how to get between any two points - and it has to work for shops that neither model has ever seen.

That drives the foundational decision: **the navigation graph is built from the official floor directory, not from our photographs**. Our survey missed some shops and mislabelled others, and a system restricted to what the camera captured could never route to a shop the survey got wrong. So every unit on a floor map becomes a node whether or not it was photographed. The models supply the ability to *recognise*; the directory supplies the world.

The payoff is concrete: **104 routable destinations across five floors instead of 75 across three**. That is why our API reports every store as routable but only some as having images - you can navigate to a shop with no reference photographs, you simply cannot use it as your starting point.

The floor model

Each floor is a rectangular walkway around a central atrium, with a top and a bottom row of shops. Shoppers cross between rows at exactly four points: the front gate at the west end, the back gate at the east end, and two escalator banks either side of the atrium. Both banks serve every floor.

I build that as a weighted graph in NetworkX - walkway nodes along each row, a short stub joining each shop to its row, the four crossing points linking the rows, and the escalator banks connected vertically between adjacent floors. Routing is Dijkstra weighted in metres, which means the router picks whichever escalator bank is nearer **automatically**. I do not code that choice; it falls out of the graph.

Modelled cost	Value	Role
Storefront frontage	8 m	Spacing along a row
Walkway to shop entrance	2 m	Stub edge
Atrium crossing	34 m	Top row to bottom row
One escalator level	25 m equivalent	Keeps routes on one floor

Be upfront that these are modelled rather than measured. Ordering and direction are reliable; the metre figures are estimates.

Turning a graph path into human directions

This is the genuinely non-trivial engineering. A raw Dijkstra path is a list of perhaps thirty nodes and nobody can follow that, so it is collapsed. Consecutive nodes on the same row become one instruction - walk toward the back gate for about forty metres. Multi-level escalator hops merge into a single step rather than one per floor. And each walk instruction **names up to three shops you pass on the way**, which is what makes an instruction verifiable while you are actually walking: you know immediately if you have gone the wrong way.

The application

A single-page app: photograph a nearby shop, see what the system recognised along with its alternatives, search for a destination, and get the route drawn leg by leg as an SVG per floor. There is

also speech-synthesis turn-by-turn guidance with auto-advance, because a shopper walking through a mall is not looking at their screen.

On production concerns: CLIP is loaded once at application startup rather than per request, otherwise every upload would pay the model-loading cost. Uploads are capped at 12 MB and re-encoded before reaching the model. The thumbnail endpoint is path-guarded so a crafted request cannot reach outside the capture folder.

Questions to expect

Why build the graph from the map rather than from your own data?

Because the map is authoritative and our survey is not. Building from the photographs meant any shop we missed simply did not exist in the app - 75 destinations instead of 104.

How does multi-floor routing choose an escalator?

It does not choose - Dijkstra does. Both banks exist on every floor and are connected vertically, so the shortest weighted path uses the nearer one. The escalator cost is set high enough that a route prefers staying on one floor when the walk is comparable.

Are the distances real?

No. They are modelled from average storefront width. Direction and ordering are trustworthy; the metre figures are estimates, and we say so.

Is this deployable as it stands?

As a demonstrator, yes - it runs end to end on a laptop and works from a phone on the same network. Production would need the label review completed, a fresh evaluation set, the missing shops photographed, and the distances calibrated against real measurements.

5. Models and evaluation - shared knowledge

Every member must be able to deliver this section. Evaluators cross-examine on it regardless of who owns which job.

The central design decision: retrieval, not classification

We deliberately framed recognition as **image retrieval** rather than classification, and neither model is trained.

A classifier needs many labelled samples per class. We have around three images per shop - at that size a classifier memorises rather than generalises. Worse, it is structurally wrong for the use case: every time the mall gains a tenant you would retrain the entire model. Retrieval needs no training at all. We embed the reference images once, and recognition is a nearest-neighbour lookup in that space.

Adding a shop means appending vectors - no retraining, no downtime. For a mall, where tenants change constantly, that is the only maintainable choice.

How the evaluation is set up

Each store with at least three images gives up exactly one as a held-out query, and that image is **never encoded into the gallery**. That yields 234 gallery vectors and 55 held-out queries.

One leakage guard worth volunteering: the per-store brand vocabulary is built from gallery frames only. Deriving it from every frame would leak the held-out query into the vocabulary it is later scored against, inflating text accuracy artificially.

We report top-1, top-3 and top-5 accuracy, plus **floor accuracy separately** - because sending a user to the wrong floor is the failure they actually notice, and it is a different kind of error from naming the wrong shop.

The ablation - the result to lead with

Configuration	Top-1	Top-3	Top-5	Floor
Visual only (CLIP)	78.0%	89.8%	93.2%	88.1%
Text only (OCR)	74.6%	93.2%	96.6%	94.9%
Fusion 0.7 / 0.3	93.3%	98.3%	98.3%	96.7%

Lead with the gap, not the absolute figure. Fusion beats the better single model by more than fifteen points. That gap is the defensible finding: it proves the two models are genuinely complementary rather than redundant. Either alone sits in the mid-to-high seventies at top-1; together they clear ninety.

On the weighting: 0.7 visual to 0.3 text was **set as the default before evaluating, not tuned afterwards on the test set**. We report the full sweep so the sensitivity is visible rather than asserted.

The two limitations to state before you are asked

Limitation one - the evaluation split is optimistic

Every held-out query was captured seconds from its own gallery images, at nearly the same angle and lighting. That is far easier than the real task. A fair number needs photographs taken on a different day in different light. Our figures are an upper bound, not a deployment estimate.

Limitation two - the labels are inferred, not verified

The benchmark grades against labels our own pipeline generated, so a genuine improvement can lower the measured score when the stored label was wrong. That is not hypothetical - it happened to us, and it is why the review tool exists. The metric becomes fully meaningful only once labelling is verified.

6. Before the viva

One inconsistency to fix

Two of our artefacts disagree, and an evaluator comparing them will notice. The dataset card is stale relative to the most recent pipeline run:

	report.md	DATASET_CARD.md
Stores	83	85
Images	289	276
Gallery / query	234 / 55	217 / 59

Job one's owner should re-run `build_dataset.py` then `tools/export_dataset.py` so both regenerate from the same state, and someone should re-run `evaluate.py` to confirm the ablation still holds under current labelling. If the numbers have moved, present the fresh ones - the argument rests on the gap between fused and single-model accuracy, not on any exact value.

Commands to have ready

Job	Command	Shows
1 - FEED	<code>python build_dataset.py</code>	Ingest, encode, dataset report
2 - FUSE	<code>python predict.py photo.jpg</code>	Recognition on one photograph
2 - FUSE	<code>python evaluate.py</code>	The ablation table
3 - ACT	<code>python api.py</code>	Full app in the browser
3 - ACT	<code>python route.py --from-photo s.jpg --to DECATHLON</code>	Photo to directions, on the CLI

What every member should be able to do

- Draw the flow: photographs, then the two models, then fusion into a recognised shop, then the navigation graph, then spoken directions.
- Name the two models, say what each reads, and explain why one alone is not enough - with the fifteen-point ablation gap as the evidence.
- State that nothing was trained, and defend retrieval over classification using the three-images-per-shop argument.
- State both limitations unprompted. Volunteering them reads as rigour; being caught by them does not.
- Answer one question about the job either side of their own, because the seams are where evaluators probe.

If you remember one line

Two models, three jobs. CLIP reads appearance, RapidOCR reads signage, and the project is the work of feeding them well, fusing them honestly, and acting on the result.